Kick Start 2021 - Round E

Analysis: Birthday Cake

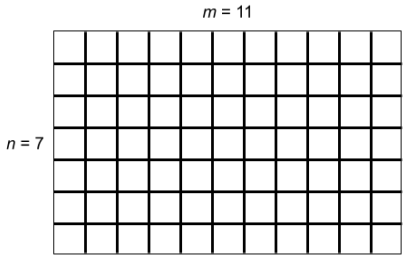
In what follows, let us suppose that the delicious part of the cake has n rows and m columns.

Test Set 1

When $\mathbf{K} = 1$, we must consider two cases. If the delicious part is fully inside the cake, meaning that the outer cells are not delicious, then all grid lines around the delicious cells must be cut. Specifically, there are $n + 1$ grid lines of length m and $m + 1$ grid lines of length n, so the answer is $n(m + 1) + m(n + 1) = 2nm + n + m$ plus the minimum distance to cut from the border to the delicious part. On the other hand, if the delicious part shares some border with the whole cake, then the answer is $2nm + n + m$ minus the length of the shared border. The time complexity is $O(1)$.

Test Set 2

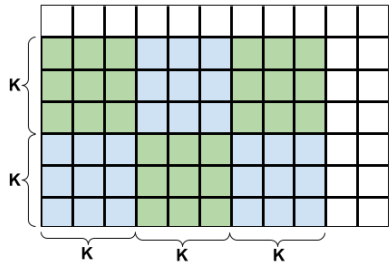
Let us start out simple. Suppose a cake of size $n \times m$ is entirely delicious and needs to be cut into 1×1 squares. This means that we need to cut the cake along all internal grid lines, which are marked bold in the following picture. Moreover, suppose for the moment that our knife is infinitely long. One way to cut the cake would be to make $n - 1$ horizontal cuts and then make $m - 1$ vertical cuts for each of the resulting n horizontal strips. This strategy amounts to $n - 1 + n(m - 1) = nm - 1$ cuts. If we start with $m - 1$ long vertical cuts first, then we would need $n - 1$ horizontal cuts for each of the m vertical strips, which again results in $m - 1 + m(n - 1) = nm - 1$ cuts.



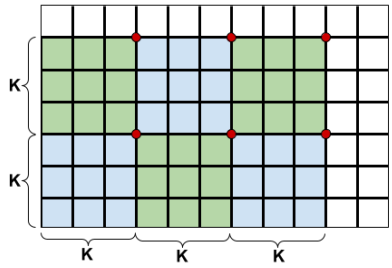
Can we do any better? No. In order to prove this claim, let us consider the worst case scenario where we make cuts of length 1 only, which means $n(m - 1) + m(n - 1) = 2nm - n - m$ cuts in total. Since we have a very long knife, we can try to combine or merge some of those unit cuts into longer cuts. Now, at each internal grid point, we can merge the two vertical cuts meeting at that point or the two horizontal cuts, but not both. Thus, we can save at most one cut at each internal grid point. There are $(n - 1)(m - 1) = nm - n - m + 1$ internal grid points, which is how many cuts we can possibly save. It follows that we need at least $2nm - n - m - (nm - n - m + 1) = nm - 1$ cuts to divide the cake into unit squares.

Now that we know how to cut a fully delicious cake with a sufficiently long knife (i.e. $\mathbf{K} \geq n$ or $\mathbf{K} \geq m$), let us turn our attention to the case where the length of the knife is $\mathbf{K} < \min(n, m)$. With a cake this large, some cuts will necessarily terminate at a previously unexposed internal grid point. For convenience, we will call them *red* points. And if we continue the reasoning from the previous paragraph, the red points will not save a cut for us. In other words, as soon as we stop at a previously unexposed grid point, the three other grid lines meeting at that point will belong to different cuts with no chance of merging any two of them. Conversely, if a cut exposes an internal grid point without terminating at it, then that grid point belongs to just three different cuts and so one cut is saved. Consequently, we want to minimize the number of red points.

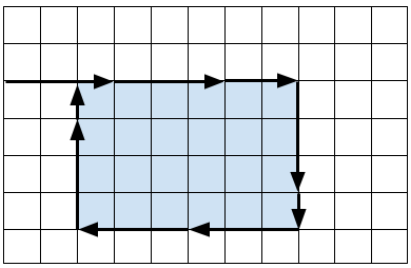
To prove a lower bound on the number of red points, let us consider $\lfloor \frac{n-1}{\mathbf{K}} \rfloor$ rows of $\mathbf{K} \times \mathbf{K}$ blocks, $\lfloor \frac{m-1}{\mathbf{K}} \rfloor$ blocks in each row as shown in the following picture for $\mathbf{K} = 3$. If we consider the blocks open at the left and bottom sides and closed at the right and top sides, then each of the blocks contains $\mathbf{K} \times \mathbf{K}$ internal grid points and any two blocks are disjoint. One can easily verify that the very first cut that has a point common with a particular block will create a red point in that block. Since the blocks are disjoint, there will be at least $\lfloor \frac{n-1}{\mathbf{K}} \rfloor \times \lfloor \frac{m-1}{\mathbf{K}} \rfloor$ red points for any cutting strategy.



It turns out that for an optimal cutting strategy this is also the upper bound on the number of red points. One way of achieving this bound is to select the top-right corner of each block as the red point, cut out the blocks with $2\lfloor \frac{n-1}{\mathbf{K}} \rfloor \times \lfloor \frac{m-1}{\mathbf{K}} \rfloor$ full cuts, and finally cut all the blocks and the L-shaped remaining part into unit squares without ever terminating at a previously unexposed grid point (i.e. without introducing new red points). Recall that every internal grid point that is not red saves us one cut. It follows that the minimum number of cuts to partition a fully delicious cake is $nm - 1 + \lfloor \frac{n-1}{\mathbf{K}} \rfloor \times \lfloor \frac{m-1}{\mathbf{K}} \rfloor$. Note that the case with a sufficiently long knife is a special case of this formula, as $\lfloor \frac{n-1}{\mathbf{K}} \rfloor \times \lfloor \frac{m-1}{\mathbf{K}} \rfloor = 0$.



It remains to solve the problem for a cake that is partially not delicious. Intuitively, it seems reasonable to cut out the delicious rectangle first and then proceed with the above strategy to partition the delicious part into unit squares. To cut out the delicious rectangle, we should start cutting from the outer border of the cake towards one of the corners of the delicious rectangle and then cut around the rectangle as shown in the following picture for $\mathbf{K} = 3$. There are eight symmetric variants to try and we take the minimum number of cuts obtained this way. We should be careful, though, not to include any sides of the delicious rectangle that are touching the border of the whole cake.



A skeptical reader might ask whether it could be worth considering a cutting strategy where we start from the outer border and cut towards somewhere in the middle of the delicious rectangle. If the distance from the border to the delicious rectangle is not divisible by $\mathbf{K}$, then the last step would cut right into the delicious part and perhaps save one cut for the internal grid lines of the

delicious rectangle. It can be shown, however, that we would need one extra cut for the outline in this case, so overall we would not gain anything. The proof of this claim is technical yet not very interesting, so it is left for the reader as an exercise.

The time complexity of the solution for Test Set 2 is still $O(1)$.

Kick Start 2021 - Round E

Analysis: Increasing Sequence Card Game

Test Set 1

For the first test set, we can generate all possible arrangements of the cards from 1 to $\mathbf{N}$. For each arrangement of the cards, we can play the game to find the score for that particular arrangement. Then the expected score of the game will be the average score over all arrangements. The total number of possible arrangements for $\mathbf{N}$ cards is $\mathbf{N}!$.

The time complexity is $O(\mathbf{N} \cdot \mathbf{N}!)$ which is under the required limit for $\mathbf{N} = 10$.

Test Set 2

Our previous strategy would not work for this test set.
Let us consider arrangements where the number on top of the pile is x. Then among the remaining $\mathbf{N} - 1$ cards, all the cards that are smaller than x will be discarded. We only need to take care of the ones that are greater than x. So, the expected score for arrangements with x on top of the pile will be one higher than the expected score of the rest of the cards greater than x. Also, it does not matter what the starting and ending number on the cards is, all that matters is the number of cards. So, the expected score of cards from $x + 1$ to $\mathbf{N}$ is equal to expected score of cards from 1 to $\mathbf{N} - x$. Assuming $E_{\mathbf{N}}$ denotes the expected score of $\mathbf{N}$ cards, the expected score when x is on top is $E_{\mathbf{N}-x} + 1$.

Now, x ranges from 1 to $\mathbf{N}$, so the expected score for $\mathbf{N}$ cards is
$E_{\mathbf{N}} = \frac{\sum_{x=1}^{\mathbf{N}}(E_{\mathbf{N}-x}+1)}{\mathbf{N}} = \frac{\sum_{i=0}^{\mathbf{N}-1} E_i}{\mathbf{N}} + 1$. The summation in the term is a cumulative sum of expected scores of first $\mathbf{N} - 1$ natural numbers. So, expected score for $\mathbf{N}$ cards can be computed in linear time if we maintain the cumulative sum of the expected score for $\mathbf{N} - 1$ cards.

Test Set 3

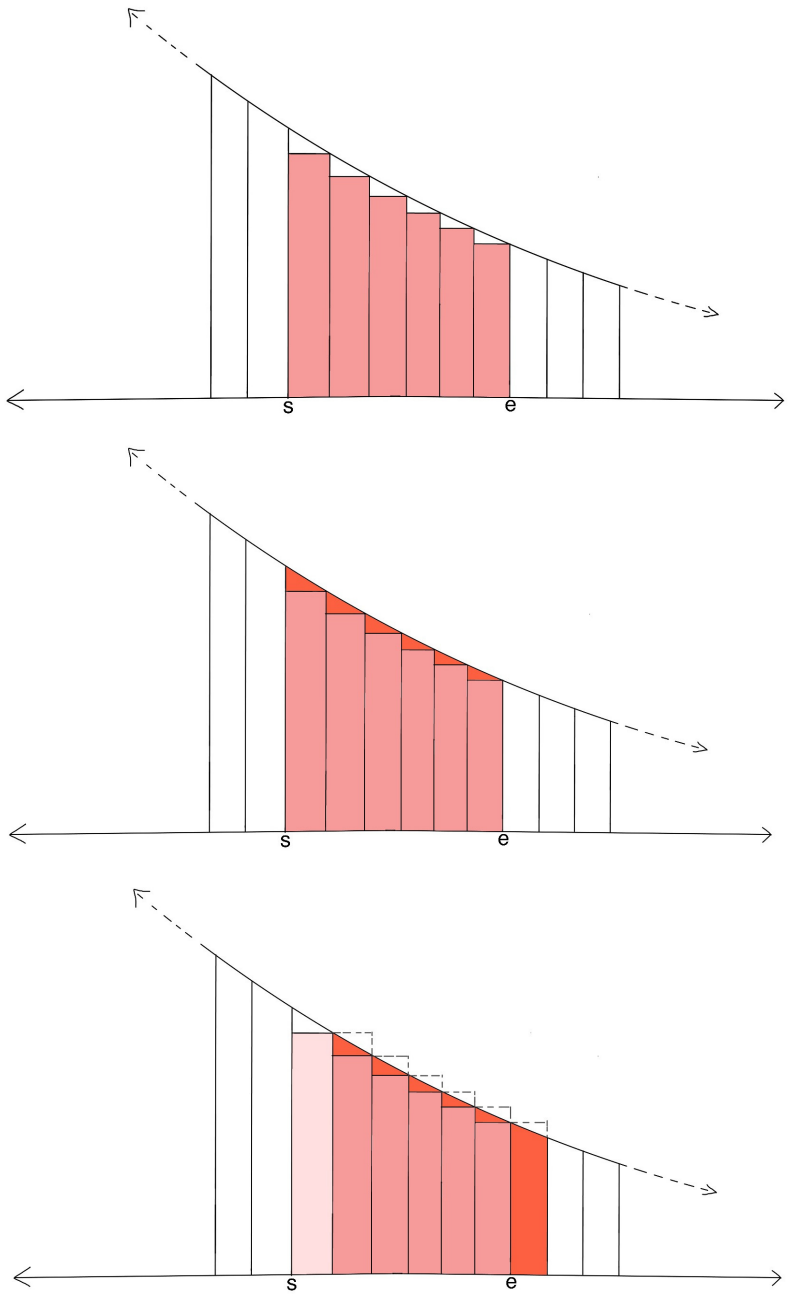
Let us denote $\sum_{i=0}^{\mathbf{N}} E_i$ as $S_{\mathbf{N}}$, then from the result of the previous section, we have
$E_{\mathbf{N}} = \frac{S_{\mathbf{N}-1}}{\mathbf{N}} + 1$. Now, for $E_{\mathbf{N}+1}$ we have:
$E_{\mathbf{N}+1} = \frac{S_{\mathbf{N}}}{\mathbf{N}+1} + 1 = \frac{E_{\mathbf{N}}+S_{\mathbf{N}-1}}{\mathbf{N}+1} + 1$
Now, substituting $E_{\mathbf{N}}$ from the previous result, we get:
$E_{\mathbf{N}+1} = \frac{\frac{S_{\mathbf{N}-1}}{\mathbf{N}}+1+S_{\mathbf{N}-1}}{\mathbf{N}+1} + 1 = \frac{\frac{(\mathbf{N}+1)S_{\mathbf{N}-1}}{\mathbf{N}}+1}{\mathbf{N}+1} + 1$
$\Rightarrow E_{\mathbf{N}+1} = \frac{S_{\mathbf{N}-1}}{\mathbf{N}} + \frac{1}{\mathbf{N}+1} + 1$
$\Rightarrow E_{\mathbf{N}+1} = E_{\mathbf{N}} + \frac{1}{\mathbf{N}+1}$

This is the harmonic series since $E_1 = 1$. We can estimate $E_{\mathbf{N}}$ for $\mathbf{N} > 10^6$ with
$E_{10^6} + \int_{10^6+1}^{\mathbf{N}+1} \frac{1}{x}\,dx$. Since $\int \frac{1}{x}\,dx = \log(x) + C$, we get (for $\mathbf{N} > 10^6$):
$E_{\mathbf{N}} = E_{10^6} + \log(\mathbf{N}+1) - \log(10^6+1)$.

We can precompute the harmonic series till 10^6 in linear time and then estimate the score using the above formula for $\mathbf{N} > 10^6$. So, the overall time complexity is constant, specifically $O(10^6)$ for the precomputation.

Error Bounds

Consider the following 3 graphs:



The first image represents the actual number we need $\sum_{x=s+1}^{e} \frac{1}{x}$, second image represents a larger area $\int_{s}^{e} \frac{1}{x}\,dx$ and third image represents a smaller area $\int_{s+1}^{e+1} \frac{1}{x}\,dx$. So, error (denoted as $Er_{s,e}$) is given by:

$$Er_{s,e} < \int_{s}^{e} \frac{1}{x}\,dx - \int_{s+1}^{e+1} \frac{1}{x}\,dx$$
$$\Rightarrow Er_{s,e} < \int_{s}^{s+1} \frac{1}{x}\,dx - \int_{e}^{e+1} \frac{1}{x}\,dx < \int_{s}^{s+1} \frac{1}{x}\,dx$$
$$\Rightarrow Er_{s,e} < \frac{1}{s}$$

Now, we are estimating only after 10^6, so $s = 10^6$, hence $Er_{10^6,e} < 10^{-6}$.

Kick Start 2021 - Round E

Analysis: Palindromic Crossword

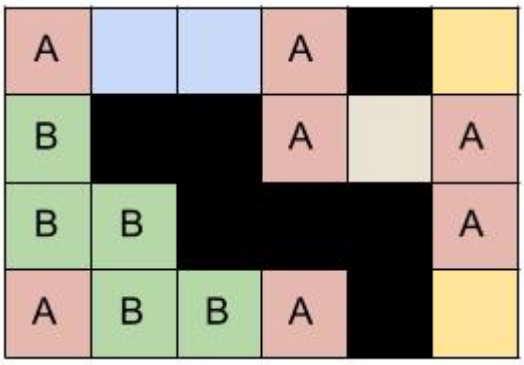
Test Set 1

Suppose we were solving the crossword on paper. Then, for each word we will check if any of the palindromic positions can be filled (we consider two positions as "palindromic positions" if their distances to the closer word boundary (start or end) are equal). We will need to repeat this process until no more positions in the crossword can be filled. We can simulate this process by repeatedly trying to fill in one more position. The simulation ends when we cannot fill any character in the crossword.

Runtime analysis: In each successful pass over the crossword, at least one character will be filled. There are at most $\mathbf{N} \times \mathbf{M}$ spots on the crossword that need to be filled, which means at most $\mathbf{N} \times \mathbf{M}$ passes are needed to fill the crossword. Now, in each pass over the crossword, we will be checking all the rows and columns once. We will need to divide each row/column into its constituent words (using '#' as separators). Then for each word, we will check the palindromic positions for a filled character and fill-in the other position. This process will take $O(\mathbf{N} \times \mathbf{M})$ for checking all rows and columns once. So, total runtime will be:
$O((\mathbf{N} \times \mathbf{M}) \times (\mathbf{N} \times \mathbf{M})) = O(\mathbf{N}^2 \times \mathbf{M}^2)$

Test Set 2

Consider the following crossword:



This image shows the equivalence classes on a partially solved crossword. Now, if we find these equivalence classes, we can fill all the cells for each class if any of the cells in that class contains a character.
Now, to build equivalence classes, let us start by building a graph from our crossword. We will consider all cells in the crossword (which are not '#') as nodes of our graph and draw edges between palindromic positions. To generate these edges, we can traverse over all rows and then over all columns of the crossword. For each row/column, split it into its constituent words and add an edge between the palindromic positions. Now, if two cells belong to the same connected component, they are equivalent. We can find connected components (equivalence classes) optimally from our constructed graph using any graph search algorithm (such as DFS or BFS) or using DSU.

Runtime analysis: Any graph traversal algorithm will visit each cell only once and a cell can be connected to at most two other cells. Then, each cell will be visited at most once more when filling in equivalence classes. So, total runtime will be:
$O(\mathbf{N} \times \mathbf{M})$

Kick Start 2021 - Round E

Analysis: Shuffled Anagrams

Test set 1

For this test set, since the length of $\mathbf{S} \leq 8$, we can try every permutation of characters and check whether there exists a permutation such that for all i, $S[i] \neq A[i]$. To find every permutation, we can first convert the string to a character array. Then, we swap the first element with every other element and recursively find permutations of the rest of the string.

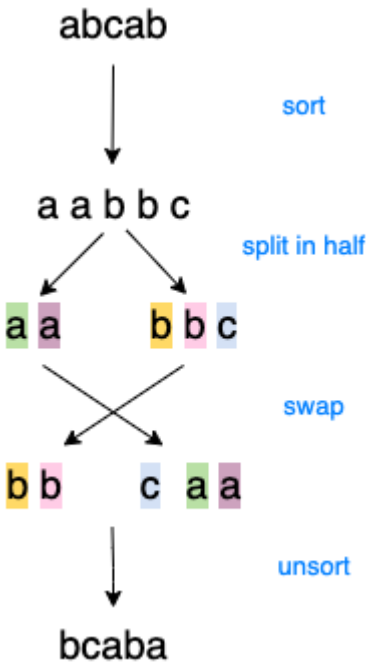
This can be performed in $O(N!)$, where N is the length of $\mathbf{S}$.

Test set 2

For this test set, the above solution would exceed the time limits.

The key observation here is that if a character exists more than $\lfloor \frac{N}{2} \rfloor$ times, then it's impossible to find such a permutation, because at least one position will have a letter that stays the same. Otherwise, we can sort the letters and keep track of the initial position of each letter.

Let the new sorted letters be P. We can split the sorted letters into two halves, from index 0 to $\frac{N}{2}$, $P[0 : \frac{N}{2}]$, and from $\frac{N}{2}$ to the end, $P[\frac{N}{2} :]$. If N is odd, split P such that the second half has an extra letter, where the first half is 0 to $\lfloor \frac{N}{2} \rfloor$ and the second half is from $\lceil \frac{N}{2} \rceil$ to the end. Then, we put each character from the second half of the sorted letters $P[i + (\frac{N}{2})]$ into the original position of the corresponding letter in the first half $P[i]$. Similarly, we put each character from the first half of the sorted letters $P[i]$ into the original position of the corresponding letter in the second half $P[i + (\frac{N}{2})]$. Note that if N is odd, the second half of the sorted letters $P[i + (\frac{N}{2})]$ will occupy the first $\lfloor \frac{N}{2} \rfloor + 1$ spaces, while the original first half will occupy the last $\lfloor \frac{N}{2} \rfloor$ spaces, as shown in the example below. The letter orginally at $P[N - 1]$ will be in the middle of the array after the swap, replacing $P[i + \lfloor \frac{N}{2} \rfloor]$.



This works because we know that no more than half the characters are equal, and hence the character at $P[i]$ cannot be equal to the letter at $P[i + (\frac{N}{2})]$.

This can be performed in $O(N \log N)$, due to sorting. However, due to the limited size of the alphabet, we can actually sort even faster using a non-comparative sorting algorithm such as counting sort.